

Behavioral Design Patterns

Five Exam-Style Problems with Complete Java Solutions

Five independent design problems, each followed by a fully worked solution

How to use this set

Attempt each problem before reading its solution. The questions are deliberately scenario-heavy: identify what varies, what must remain stable, and which objects should own transitions, algorithms, coordination, workflow order, or notifications.

No.	Scenario	Main difficulty
1	Research Grant Proposal Lifecycle	cyclic and terminal transitions
2	Adaptive Delivery Dispatch Planner	runtime overrides and tie-breaking
3	Laboratory Equipment Borrowing Coordinator	role plus stage authorization
4	Online Assessment Processing Pipeline	fixed algorithm with hooks
5	Preference-Aware Transit Disruption Alerts	filtered, stable notification

PROBLEM 1

Research Grant Proposal Lifecycle

A university research office manages grant proposals. The operations allowed on a proposal depend on its current condition, and the same operation may have a different effect - or be forbidden - after the proposal moves to another condition.

Lifecycle rules

- Draft: the title may be edited repeatedly. A nonblank proposal may be submitted or withdrawn.
- Submitted: its contents are locked. The research office may assign a reviewer, moving it to Under Review. The investigator may still withdraw it before review begins.
- Under Review: the reviewer may request a revision, approve, or reject. Withdrawal and direct editing are no longer permitted.
- Revision Requested: the investigator may edit and resubmit. Each resubmission returns the proposal to Submitted, so a reviewer must be assigned again. It may also be withdrawn.
- Approved, Rejected, and Withdrawn: these are final conditions. No later action may change the proposal.

Required operations

- editTitle(String title)
- submit()
- assignReviewer(String reviewer)
- requestRevision(String feedback)
- approve(), reject(), and withdraw()

Important behavior

- Invalid operations must display a meaningful message and leave all data and the current condition unchanged.
- A blank title cannot be saved, a blank reviewer cannot be assigned, and revision feedback cannot be blank.
- The implementation should avoid a large condition-dependent method in the proposal class and should allow another proposal condition to be added with minimal change.

Task

1. Design and implement the complete lifecycle.
2. Keep condition-specific behavior modular and perform every valid transition.
3. Demonstrate a revision cycle, a successful approval, and at least one invalid operation.

Exam instruction

You may introduce suitable interfaces, classes, enumerations, records, and helper methods. Your solution should be clean, object-oriented, and open to reasonable extension.

SOLUTION | STATE

Design rationale

Why this pattern fits

The proposal behaves differently as its internal condition changes. Each condition is represented by a separate state object; the Proposal delegates operations to that object and remains the stable context. Terminal states inherit the default rejection behavior.

Pattern-role mapping

Pattern role	Implementation
Context	Proposal - owns data and the current ProposalState
State interface	ProposalState - declares every condition-dependent operation
Concrete states	DraftState, SubmittedState, UnderReviewState, RevisionRequestedState, and terminal states
Transitions	Concrete states call Proposal.changeState(...) after a valid action

Complete Java solution - GrantProposalDemo.java

```
public class GrantProposalDemo {
    interface ProposalState {
        default void editTitle(Proposal p, String title) { invalid(p, "edit title"); }
        default void submit(Proposal p) { invalid(p, "submit"); }
        default void assignReviewer(Proposal p, String reviewer) {
            invalid(p, "assign reviewer");
        }
        default void requestRevision(Proposal p, String feedback) {
            invalid(p, "request revision");
        }
        default void approve(Proposal p) { invalid(p, "approve"); }
        default void reject(Proposal p) { invalid(p, "reject"); }
        default void withdraw(Proposal p) { invalid(p, "withdraw"); }
        String name();
    }

    private static void invalid(Proposal p, String action) {
        System.out.println("Cannot " + action + " while proposal is "
            + p.stateName() + ".");
    }
}

static final class Proposal {
    private String title;
    private String reviewer;
    private String feedback;
    private ProposalState state = new DraftState();

    Proposal(String title) { this.title = title; }

    void changeState(ProposalState next) {
        System.out.println("STATE: " + state.name() + " -> " + next.name());
        state = next;
    }

    String stateName() { return state.name(); }
    String title() { return title; }
    void setTitle(String value) { title = value; }
    void setReviewer(String value) { reviewer = value; }
    void setFeedback(String value) { feedback = value; }

    void editTitle(String value) { state.editTitle(this, value); }
    void submit() { state.submit(this); }
    void assignReviewer(String value) { state.assignReviewer(this, value); }
    void requestRevision(String value) { state.requestRevision(this, value); }
    void approve() { state.approve(this); }
    void reject() { state.reject(this); }
    void withdraw() { state.withdraw(this); }
}

static final class DraftState implements ProposalState {
    public String name() { return "DRAFT"; }

    public void editTitle(Proposal p, String title) {
        if (title == null || title.isBlank()) {
            System.out.println("Title cannot be blank.");
            return;
        }
        p.setTitle(title);
        System.out.println("Title updated.");
    }

    public void submit(Proposal p) {
        if (p.title() == null || p.title().isBlank()) {
            System.out.println("A title is required before submission.");
            return;
        }
    }
}
```

```

        }
        p.changeState(new SubmittedState());
    }

    public void withdraw(Proposal p) {
        p.changeState(new WithdrawnState());
    }
}

static final class SubmittedState implements ProposalState {
    public String name() { return "SUBMITTED"; }

    public void assignReviewer(Proposal p, String reviewer) {
        if (reviewer == null || reviewer.isBlank()) {
            System.out.println("Reviewer name cannot be blank.");
            return;
        }
        p.setReviewer(reviewer);
        p.changeState(new UnderReviewState());
    }

    public void withdraw(Proposal p) {
        p.changeState(new WithdrawnState());
    }
}

static final class UnderReviewState implements ProposalState {
    public String name() { return "UNDER_REVIEW"; }

    public void requestRevision(Proposal p, String feedback) {
        if (feedback == null || feedback.isBlank()) {
            System.out.println("Revision feedback is required.");
            return;
        }
        p.setFeedback(feedback);
        p.changeState(new RevisionRequestedState());
    }

    public void approve(Proposal p) { p.changeState(new ApprovedState()); }
    public void reject(Proposal p) { p.changeState(new RejectedState()); }
}

static final class RevisionRequestedState implements ProposalState {
    public String name() { return "REVISION_REQUESTED"; }

    public void editTitle(Proposal p, String title) {
        if (title == null || title.isBlank()) {
            System.out.println("Title cannot be blank.");
            return;
        }
        p.setTitle(title);
        System.out.println("Revised title saved.");
    }

    public void submit(Proposal p) {
        p.changeState(new SubmittedState());
    }

    public void withdraw(Proposal p) {
        p.changeState(new WithdrawnState());
    }
}

abstract static class TerminalState implements ProposalState {
    private final String name;
    TerminalState(String name) { this.name = name; }
    public String name() { return name; }
}

static final class ApprovedState extends TerminalState {
    ApprovedState() { super("APPROVED"); }
}
static final class RejectedState extends TerminalState {
    RejectedState() { super("REJECTED"); }
}
static final class WithdrawnState extends TerminalState {
    WithdrawnState() { super("WITHDRAWN"); }
}

public static void main(String[] args) {
    Proposal p = new Proposal("AI-assisted testing");
    p.submit();
    p.editTitle("Illegal edit");
    p.assignReviewer("Dr. Rahman");
    p.requestRevision("Clarify the evaluation plan.");
    p.editTitle("AI-assisted testing: revised");
    p.submit();
    p.assignReviewer("Dr. Rahman");
    p.approve();
    p.withdraw();           // rejected: APPROVED is terminal
}
}

```

Implementation check

This listing is self-contained and uses one public demo class. Save it under the filename GrantProposalDemo.java, then compile and run it with Java 17 or later.

PROBLEM 2

Adaptive Delivery Dispatch Planner

A logistics company maintains a queue of delivery jobs. Each job has an ID, execution duration, deadline, estimated cost, handling-risk score, fragile flag, and priority. The planner has a user-selected preferred policy, but workload rules may temporarily override it before every dispatch.

Available dispatch policies

- Economy: choose the lowest estimated cost; break ties by earlier deadline, then ID.
- Express: choose the smallest current slack, where $\text{slack} = \text{deadline} - \text{current time} - \text{duration}$; break ties by HIGH, MEDIUM, LOW priority, then deadline and ID.
- Fragile Safe: fragile jobs precede nonfragile jobs; then choose lower risk, earlier deadline, and ID.

Adaptive override rules

- Urgent workload: if any waiting job has slack less than or equal to 2, use Express regardless of the preferred policy.
- Fragile workload: if there is no urgent job but at least three fragile jobs wait, use Fragile Safe.
- Normal workload: otherwise use the current preferred policy.

Time and reevaluation

- The clock begins at a constructor-supplied time. After a job is dispatched, advance the clock by that job's duration.
- Reevaluate the override rules before every next dispatch. A previously safe job may become urgent as time advances.
- Changing the preferred policy must not replace the planner object or discard queued jobs.

Required operations

- `addJob(...)`, `setPreferredPolicy(...)`, `dispatchNext()`, and `dispatchAll()`
- Reject duplicate job IDs. `dispatchNext()` displays the chosen job, active policy, current time, and slack.

Task

1. Implement all three interchangeable selection policies.
2. Implement the adaptive selection rules in the planner without embedding policy-specific sorting logic there.
3. Demonstrate at least one temporary override and a runtime preferred-policy change.

Exam instruction

You may introduce suitable interfaces, classes, enumerations, records, and helper methods. Your solution should be clean, object-oriented, and open to reasonable extension.

SOLUTION | STRATEGY

Design rationale

Why this pattern fits

The selection algorithm is the changing part. DispatchStrategy encapsulates that algorithm, while DispatchPlanner owns the queue and decides which strategy is active. The adaptive rule chooses among strategy objects; it does not duplicate their logic.

Pattern-role mapping

Pattern role	Implementation
Context	DispatchPlanner - queue, clock, preferred strategy, and override rules
Strategy	DispatchStrategy - select(...) and name()
Concrete strategies	EconomyStrategy, ExpressStrategy, and FragileSafeStrategy
Data	Immutable DeliveryJob record; slack is computed for the current clock

Complete Java solution - AdaptiveDispatchDemo.java

```
import java.util.*;

public class AdaptiveDispatchDemo {
    enum Priority { HIGH, MEDIUM, LOW }

    record DeliveryJob(
        String id, int duration, int deadline, double cost,
        int risk, boolean fragile, Priority priority) {
        int slackAt(int now) { return deadline - now - duration; }
    }

    interface DispatchStrategy {
        DeliveryJob select(List<DeliveryJob> jobs, int now);
        String name();
    }

    static final class EconomyStrategy implements DispatchStrategy {
        public String name() { return "ECONOMY"; }
        public DeliveryJob select(List<DeliveryJob> jobs, int now) {
            return jobs.stream().min(Comparator
                .comparingDouble(DeliveryJob::cost)
                .thenComparingInt(DeliveryJob::deadline)
                .thenComparing(DeliveryJob::id)).orElseThrow();
        }
    }

    static final class ExpressStrategy implements DispatchStrategy {
        public String name() { return "EXPRESS"; }
        public DeliveryJob select(List<DeliveryJob> jobs, int now) {
            return jobs.stream().min(Comparator
                .comparingInt((DeliveryJob j) -> j.slackAt(now))
                .thenComparing((DeliveryJob j) -> j.priority().ordinal())
                .thenComparingInt(DeliveryJob::deadline)
                .thenComparing(DeliveryJob::id)).orElseThrow();
        }
    }

    static final class FragileSafeStrategy implements DispatchStrategy {
        public String name() { return "FRAGILE_SAFE"; }
        public DeliveryJob select(List<DeliveryJob> jobs, int now) {
            return jobs.stream().min(Comparator
                .comparing((DeliveryJob j) -> !j.fragile())
                .thenComparingInt(DeliveryJob::risk)
                .thenComparingInt(DeliveryJob::deadline)
                .thenComparing(DeliveryJob::id)).orElseThrow();
        }
    }

    static final class DispatchPlanner {
        private final List<DeliveryJob> waiting = new ArrayList<>();
        private final DispatchStrategy express = new ExpressStrategy();
        private final DispatchStrategy fragileSafe = new FragileSafeStrategy();
        private DispatchStrategy preferred;
        private int now;

        DispatchPlanner(DispatchStrategy preferred, int startTime) {
            this.preferred = Objects.requireNonNull(preferred);
            this.now = startTime;
        }

        void addJob(DeliveryJob job) {
            if (waiting.stream().anyMatch(j -> j.id().equals(job.id()))) {
                throw new IllegalArgumentException("Duplicate job ID: " + job.id());
            }
            waiting.add(job);
        }
    }
}
```

```

    }

    void setPreferredPolicy(DispatchStrategy strategy) {
        preferred = Objects.requireNonNull(strategy);
        System.out.println("Preferred strategy is now " + preferred.name());
    }

    private DispatchStrategy chooseCurrentStrategy() {
        boolean urgent = waiting.stream().anyMatch(j -> j.slackAt(now) <= 2);
        if (urgent) return express;

        long fragileCount = waiting.stream().filter(DeliveryJob::fragile).count();
        if (fragileCount >= 3) return fragileSafe;

        return preferred;
    }

    DeliveryJob dispatchNext() {
        if (waiting.isEmpty()) {
            System.out.println("No delivery jobs are waiting.");
            return null;
        }
        DispatchStrategy active = chooseCurrentStrategy();
        DeliveryJob selected = active.select(List.copyOf(waiting), now);
        waiting.remove(selected);
        System.out.printf(
            "t=%d | %-12s | dispatched %s | slack=%d\n",
            now, active.name(), selected.id(), selected.slackAt(now));
        now += selected.duration();
        return selected;
    }

    void dispatchAll() {
        while (!waiting.isEmpty()) dispatchNext();
    }
}

public static void main(String[] args) {
    Dispatcher planner = new Dispatcher(new EconomyStrategy(), 0);
    planner.addJob(new DeliveryJob(
        "D1", 4, 20, 40, 2, true, Priority.MEDIUM));
    planner.addJob(new DeliveryJob(
        "D2", 3, 18, 22, 1, true, Priority.LOW));
    planner.addJob(new DeliveryJob(
        "D3", 2, 16, 30, 3, true, Priority.HIGH));
    planner.addJob(new DeliveryJob(
        "D4", 5, 30, 10, 5, false, Priority.LOW));
    planner.addJob(new DeliveryJob(
        "D5", 3, 5, 55, 2, false, Priority.HIGH));

    planner.dispatchNext(); // urgent D5 forces EXPRESS
    planner.setPreferredPolicy(new EconomyStrategy());
    planner.dispatchAll(); // decision is recomputed before every job
}

```

Implementation check

This listing is self-contained and uses one public demo class. Save it under the filename `AdaptiveDispatchDemo.java`, then compile and run it with Java 17 or later.

PROBLEM 3

Laboratory Equipment Borrowing Coordinator

A university laboratory processes equipment-borrowing requests involving a student, supervisor, equipment officer, and accounts officer. These participants must not call one another directly. A central coordinator enforces the order of work and sends the right notifications.

Workflow

- A student submits a unique request. The supervisor may approve or reject it.
- After approval, the equipment officer confirms availability or marks the item unavailable. Unavailability rejects the request.
- After availability is confirmed, Accounts clears the refundable deposit. Only then may the equipment officer issue the item.
- Only the student who owns the request may return it. The equipment officer then inspects it.
- If inspection is satisfactory, the request completes and Accounts is told to release the deposit. If damage is reported, Accounts must record payment of the damage charge before completion.

Coordination rules

- Every event is accepted only from the responsible role and only at its valid workflow stage.
- An invalid event must not alter the request and must produce a message to the sender.
- Participants know the coordinator but have no references to other participants.
- The coordinator stores multiple requests by unique ID and routes notifications to registered roles.

Required operations

- `register(...)`, `submit(...)`, and `handle(sender, requestId, event)`
- Support `approve/reject`, `availability result`, `deposit clearance`, `issue`, `return`, `inspection result`, and `damage-charge settlement`.

Task

1. Design and implement the participants, workflow data, events, and central coordinator.
2. Enforce both stage order and sender authorization.
3. Demonstrate a damaged-return path and an out-of-order action that is blocked.

Exam instruction

You may introduce suitable interfaces, classes, enumerations, records, and helper methods. Your solution should be clean, object-oriented, and open to reasonable extension.

SOLUTION | MEDIATOR

Design rationale

Why this pattern fits

The difficult part is the web of communication, so it is centralized. Participants send events only to BorrowMediator. LabBorrowMediator validates stage and role, changes the request, and routes notifications, leaving colleague classes independent.

Pattern-role mapping

Pattern role	Implementation
Mediator interface	BorrowMediator - registration, submission, and event handling
Concrete mediator	LabBorrowMediator - workflow enforcement and notification routing
Colleagues	Student, Supervisor, EquipmentOfficer, and AccountsOfficer
Workflow message	Event enum plus request ID; avoids fragile string comparisons

Complete Java solution - EquipmentBorrowingDemo.java

```
import java.util.*;

public class EquipmentBorrowingDemo {
    enum Role { STUDENT, SUPERVISOR, EQUIPMENT_OFFICER, ACCOUNTS }
    enum Stage {
        SUBMITTED, SUPERVISOR_APPROVED, AVAILABLE, DEPOSIT_CLEARED,
        ISSUED, RETURNED, DAMAGE_REPORTED, COMPLETED, REJECTED
    }
    enum Event {
        APPROVE, REJECT, CONFIRM_AVAILABLE, MARK_UNAVAILABLE,
        CLEAR_DEPOSIT, ISSUE, RETURN_ITEM, INSPECT_OK,
        INSPECT_DAMAGED, DAMAGE_CHARGE_PAID
    }

    static final class BorrowRequest {
        final String id;
        final String studentName;
        final String equipment;
        Stage stage = Stage.SUBMITTED;

        BorrowRequest(String id, String studentName, String equipment) {
            this.id = id;
            this.studentName = studentName;
            this.equipment = equipment;
        }
    }

    interface BorrowMediator {
        void register(Participant participant);
        void submit(Student student, String requestId, String equipment);
        void handle(Participant sender, String requestId, Event event);
    }

    abstract static class Participant {
        protected final String name;
        protected final Role role;
        protected BorrowMediator mediator;

        Participant(String name, Role role) {
            this.name = name;
            this.role = role;
        }

        void connect(BorrowMediator mediator) {
            this.mediator = mediator;
            mediator.register(this);
        }

        void send(String requestId, Event event) {
            mediator.handle(this, requestId, event);
        }

        void receive(String message) {
            System.out.println(role + " " + name + " <- " + message);
        }
    }

    static final class Student extends Participant {
        Student(String name) { super(name, Role.STUDENT); }
        void submit(String id, String equipment) {
            mediator.submit(this, id, equipment);
        }
    }

    static final class Supervisor extends Participant {
        Supervisor(String name) { super(name, Role.SUPERVISOR); }
    }
}
```

```

}
static final class EquipmentOfficer extends Participant {
    EquipmentOfficer(String name) { super(name, Role.EQUIPMENT_OFFICER); }
}
static final class AccountsOfficer extends Participant {
    AccountsOfficer(String name) { super(name, Role.ACCOUNTS); }
}

static final class LabBorrowMediator implements BorrowMediator {
    private final Map<Role, Participant> participants = new EnumMap<>(Role.class);
    private final Map<String, BorrowRequest> requests = new LinkedHashMap<>();

    public void register(Participant p) {
        if (participants.putIfAbsent(p.role, p) != null) {
            throw new IllegalStateException("A participant already has role " + p.role);
        }
    }

    public void submit(Student student, String id, String equipment) {
        requireRegistered(student);
        if (id == null || id.isBlank() || equipment == null || equipment.isBlank()) {
            throw new IllegalArgumentException("ID and equipment are required.");
        }
        if (requests.containsKey(id)) {
            throw new IllegalArgumentException("Duplicate request ID: " + id);
        }
        BorrowRequest r = new BorrowRequest(id, student.name, equipment);
        requests.put(id, r);
        notify(Role.SUPERVISOR, id + " awaits supervisor approval.");
        student.receive(id + " submitted for " + equipment + ".");
    }

    public void handle(Participant sender, String id, Event event) {
        requireRegistered(sender);
        BorrowRequest r = requests.get(id);
        if (r == null) {
            sender.receive("Unknown request: " + id);
            return;
        }

        boolean accepted = switch (r.stage) {
            case SUBMITTED -> submitted(sender, r, event);
            case SUPERVISOR_APPROVED -> supervisorApproved(sender, r, event);
            case AVAILABLE -> available(sender, r, event);
            case DEPOSIT_CLEARED -> depositCleared(sender, r, event);
            case ISSUED -> issued(sender, r, event);
            case RETURNED -> returned(sender, r, event);
            case DAMAGE_REPORTED -> damageReported(sender, r, event);
            case COMPLETED, REJECTED -> false;
        };
        if (!accepted) {
            sender.receive("Action " + event + " is invalid while " + id
                + " is " + r.stage + ".");
        }
    }

    private boolean submitted(Participant s, BorrowRequest r, Event e) {
        if (s.role != Role.SUPERVISOR) return false;
        if (e == Event.APPROVE) {
            move(r, Stage.SUPERVISOR_APPROVED);
            notify(Role.EQUIPMENT_OFFICER, r.id + " needs availability check.");
            return true;
        }
        if (e == Event.REJECT) {
            move(r, Stage.REJECTED);
            notify(Role.STUDENT, r.id + " was rejected by the supervisor.");
            return true;
        }
        return false;
    }

    private boolean supervisorApproved(Participant s, BorrowRequest r, Event e) {
        if (s.role != Role.EQUIPMENT_OFFICER) return false;
        if (e == Event.CONFIRM_AVAILABLE) {
            move(r, Stage.AVAILABLE);
            notify(Role.ACCOUNTS, r.id + " awaits deposit clearance.");
            return true;
        }
        if (e == Event.MARK_UNAVAILABLE) {
            move(r, Stage.REJECTED);
            notify(Role.STUDENT, r.id + " rejected: equipment unavailable.");
            return true;
        }
        return false;
    }

    private boolean available(Participant s, BorrowRequest r, Event e) {
        if (s.role == Role.ACCOUNTS && e == Event.CLEAR_DEPOSIT) {
            move(r, Stage.DEPOSIT_CLEARED);
            notify(Role.EQUIPMENT_OFFICER, r.id + " is ready for issue.");
            return true;
        }
        return false;
    }

    private boolean depositCleared(Participant s, BorrowRequest r, Event e) {
        if (s.role == Role.EQUIPMENT_OFFICER && e == Event.ISSUE) {
            move(r, Stage.ISSUED);
            notify(Role.STUDENT, r.id + ": collect " + r.equipment + ".");
            return true;
        }
    }

```

```

        }
        return false;
    }

    private boolean issued(Participant s, BorrowRequest r, Event e) {
        if (s.role == Role.STUDENT && e == Event.RETURN_ITEM
            && s.name.equals(r.studentName)) {
            move(r, Stage.RETURNED);
            notify(Role.EQUIPMENT_OFFICER, r.id + " awaits return inspection.");
            return true;
        }
        return false;
    }

    private boolean returned(Participant s, BorrowRequest r, Event e) {
        if (s.role != Role.EQUIPMENT_OFFICER) return false;
        if (e == Event.INSPECT_OK) {
            move(r, Stage.COMPLETED);
            notify(Role.ACCOUNTS, r.id + ": release refundable deposit.");
            notify(Role.STUDENT, r.id + " completed successfully.");
            return true;
        }
        if (e == Event.INSPECT_DAMAGED) {
            move(r, Stage.DAMAGE_REPORTED);
            notify(Role.ACCOUNTS, r.id + ": collect damage charge.");
            return true;
        }
        return false;
    }

    private boolean damageReported(Participant s, BorrowRequest r, Event e) {
        if (s.role == Role.ACCOUNTS && e == Event.DAMAGE_CHARGE_PAID) {
            move(r, Stage.COMPLETED);
            notify(Role.STUDENT, r.id + " closed after damage settlement.");
            return true;
        }
        return false;
    }

    private void move(BorrowRequest r, Stage next) {
        System.out.println(r.id + ": " + r.stage + " -> " + next);
        r.stage = next;
    }

    private void notify(Role role, String message) {
        Participant p = participants.get(role);
        if (p != null) p.receive(message);
    }

    private void requireRegistered(Participant p) {
        if (participants.get(p.role) != p) {
            throw new IllegalStateException(p.name + " is not registered.");
        }
    }
}

public static void main(String[] args) {
    LabBorrowMediator mediator = new LabBorrowMediator();
    Student student = new Student("Nadia");
    Supervisor supervisor = new Supervisor("Dr. Karim");
    EquipmentOfficer lab = new EquipmentOfficer("Mr. Alam");
    AccountsOfficer accounts = new AccountsOfficer("Ms. Sultana");
    student.connect(mediator);
    supervisor.connect(mediator);
    lab.connect(mediator);
    accounts.connect(mediator);

    student.submit("BR-17", "Oscilloscope");
    lab.send("BR-17", Event.ISSUE); // invalid and blocked
    supervisor.send("BR-17", Event.APPROVE);
    lab.send("BR-17", Event.CONFIRM_AVAILABLE);
    accounts.send("BR-17", Event.CLEAR_DEPOSIT);
    lab.send("BR-17", Event.ISSUE);
    student.send("BR-17", Event.RETURN_ITEM);
    lab.send("BR-17", Event.INSPECT_DAMAGED);
    accounts.send("BR-17", Event.DAMAGE_CHARGE_PAID);
}
}

```

Implementation check

This listing is self-contained and uses one public demo class. Save it under the filename `EquipmentBorrowingDemo.java`, then compile and run it with Java 17 or later.

PROBLEM 4

Online Assessment Processing Pipeline

A course platform processes programming assignments, written reports, and presentations. All assessment types must follow the same high-level sequence, but artifact validation, integrity checking, marking, and score adjustment vary by type. The sequence must never be accidentally reordered by a subtype.

Invariant processing order

- Validate common data, then validate the type-specific artifact.
- Run an integrity check when required, run an optional pre-marking step, compute the core mark, apply type-specific adjustment, clamp the result to 0..100, save it, and notify the student.
- If validation or integrity checking fails, stop immediately: do not mark, save, or notify a score.

Programming assignment

- Accept only JAVA or CPP, require successful compilation and valid test counts.
- Core mark = 80% automated-test percentage + 20% code-quality score.
- Run similarity checking with a 30% limit. Deduct 2 marks per late hour, capped at 20 marks.

Written report

- Require PDF, 5 to 20 pages, at least three citations, and no lateness.
- The rubric score is the core mark. Similarity must not exceed 25%.

Presentation

- Require a SLIDES artifact containing 5 to 15 slides, recorded attendance, and no lateness.
- Core mark = 60% deck rubric + 40% delivery. Skip similarity checking and log attendance before marking.
- Add a nonnegative question-answer bonus capped at 5 marks.

Task

1. Implement one non-overridable processing algorithm and the three assessment variants.
2. Use primitive operations and hooks so variants change only the necessary steps.
3. Demonstrate successful processing of all three valid sample submissions.

Exam instruction

You may introduce suitable interfaces, classes, enumerations, records, and helper methods. Your solution should be clean, object-oriented, and open to reasonable extension.

**SOLUTION | TEMPLATE
METHOD**

Design rationale

Why this pattern fits

The overall algorithm is fixed but selected steps vary. `AssessmentProcessor.process()` is final and defines the skeleton. Abstract methods are mandatory variant steps; default methods such as `beforeMarking()`, `adjustScore()`, and `requiresIntegrityCheck()` are hooks.

Pattern-role mapping

Pattern role	Implementation
Abstract class	<code>AssessmentProcessor</code> - owns the final <code>process(...)</code> template
Primitive operations	<code>validateArtifact(...)</code> and <code>markCore(...)</code>
Hooks	<code>requiresIntegrityCheck()</code> , <code>beforeMarking()</code> , <code>adjustScore()</code> , and <code>allowedSimilarity()</code>
Concrete classes	<code>ProgrammingProcessor</code> , <code>ReportProcessor</code> , and <code>PresentationProcessor</code>

Complete Java solution - `AssessmentTemplateDemo.java`

```
public class AssessmentTemplateDemo {
    record Submission(
        String studentId, int hoursLate, String artifactType,
        int pagesOrSlides, int citations, double similarity,
        boolean compiles, int testsPassed, int totalTests,
        double quality, double rubric, double delivery,
        boolean presented, double questionBonus) {}

    abstract static class AssessmentProcessor {
        public final double process(Submission s) {
            System.out.println("-- Processing " + s.studentId() + " --");
            validateCommon(s);
            validateArtifact(s);
            if (requiresIntegrityCheck()) checkIntegrity(s);
            beforeMarking(s); // optional hook
            double score = markCore(s); // variable step
            score = adjustScore(s, score); // optional hook
            score = Math.max(0, Math.min(100, score));
            saveResult(s, score);
            notifyStudent(s, score);
            return score;
        }

        private void validateCommon(Submission s) {
            if (s.studentId() == null || s.studentId().isBlank()) {
                fail("Student ID is required.");
            }
            if (s.hoursLate() < 0) fail("Late hours cannot be negative.");
        }

        protected abstract void validateArtifact(Submission s);
        protected abstract double markCore(Submission s);

        protected boolean requiresIntegrityCheck() { return true; }

        protected void checkIntegrity(Submission s) {
            if (s.similarity() > allowedSimilarity()) {
                fail("Similarity " + s.similarity() + "% exceeds the limit.");
            }
        }

        protected double allowedSimilarity() { return 30.0; }
        protected void beforeMarking(Submission s) {}
        protected double adjustScore(Submission s, double score) { return score; }

        private void saveResult(Submission s, double score) {
            System.out.printf("Saved result for %s: %.2f\n", s.studentId(), score);
        }

        private void notifyStudent(Submission s, double score) {
            System.out.printf("Notification sent to %s (score %.2f).\n",
                s.studentId(), score);
        }

        protected final void fail(String message) {
            throw new IllegalArgumentException(message);
        }
    }

    static final class ProgrammingProcessor extends AssessmentProcessor {
        protected void validateArtifact(Submission s) {
            if (!"JAVA".equalsIgnoreCase(s.artifactType())
                && !"CPP".equalsIgnoreCase(s.artifactType())) {

```

```

        fail("Only JAVA or CPP source is accepted.");
    }
    if (!s.compiles()) fail("Submission does not compile.");
    if (s.totalTests() <= 0 || s.testsPassed() < 0
        || s.testsPassed() > s.totalTests()) {
        fail("Invalid test counts.");
    }
    if (s.quality() < 0 || s.quality() > 100) {
        fail("Quality score must be within 0..100.");
    }
}

protected double markCore(Submission s) {
    double tests = 100.0 * s.testsPassed() / s.totalTests();
    return 0.80 * tests + 0.20 * s.quality();
}

protected double adjustScore(Submission s, double score) {
    return score - Math.min(20, 2 * s.hoursLate());
}
}

static final class ReportProcessor extends AssessmentProcessor {
    protected void validateArtifact(Submission s) {
        if (!"PDF".equalsIgnoreCase(s.artifactType())) {
            fail("The report must be a PDF.");
        }
        if (s.pagesOrSlides() < 5 || s.pagesOrSlides() > 20) {
            fail("The report must contain 5 to 20 pages.");
        }
        if (s.citations() < 3) fail("At least three citations are required.");
        if (s.hoursLate() > 0) fail("Late reports are not accepted.");
        if (s.rubric() < 0 || s.rubric() > 100) {
            fail("Rubric score must be within 0..100.");
        }
    }

    protected double markCore(Submission s) { return s.rubric(); }
    protected double allowedSimilarity() { return 25.0; }
}

static final class PresentationProcessor extends AssessmentProcessor {
    protected void validateArtifact(Submission s) {
        if (!"SLIDES".equalsIgnoreCase(s.artifactType())) {
            fail("Presentation slides are required.");
        }
        if (s.pagesOrSlides() < 5 || s.pagesOrSlides() > 15) {
            fail("The deck must contain 5 to 15 slides.");
        }
        if (!s.presented()) fail("The student did not present.");
        if (s.hoursLate() > 0) fail("Late slide decks are not accepted.");
        if (s.rubric() < 0 || s.rubric() > 100
            || s.delivery() < 0 || s.delivery() > 100) {
            fail("Deck and delivery scores must be within 0..100.");
        }
    }

    protected boolean requiresIntegrityCheck() { return false; }

    protected void beforeMarking(Submission s) {
        System.out.println("Presentation attendance confirmed.");
    }

    protected double markCore(Submission s) {
        return 0.60 * s.rubric() + 0.40 * s.delivery();
    }

    protected double adjustScore(Submission s, double score) {
        return score + Math.min(5, Math.max(0, s.questionBonus()));
    }
}

public static void main(String[] args) {
    Submission code = new Submission(
        "2305003", 3, "JAVA", 0, 0, 12,
        true, 18, 20, 85, 0, 0, false, 0);
    Submission report = new Submission(
        "2305011", 0, "PDF", 12, 8, 18,
        false, 0, 0, 0, 88, 0, false, 0);
    Submission slides = new Submission(
        "2305027", 0, "SLIDES", 10, 0, 0,
        false, 0, 0, 0, 82, 90, true, 4);

    new ProgrammingProcessor().process(code);
    new ReportProcessor().process(report);
    new PresentationProcessor().process(slides);
}
}

```

Implementation check

This listing is self-contained and uses one public demo class. Save it under the filename `AssessmentTemplateDemo.java`, then compile and run it with Java 17 or later.

PROBLEM 5

Preference-Aware Transit Disruption Alerts

A transit-control center publishes disruption alerts. Commuters choose which routes, alert types, and minimum severity they care about. Subscriptions may change while the system runs, and publishers must remain independent of concrete commuter classes.

Alert and preference data

- Each alert has a unique ID, route, type (Delay, Cancellation, Route Change, or Safety), severity (Info, Warning, or Critical), and message.
- A preference contains a set of routes, a set of alert types, and a minimum severity. Route '*' means every route.
- An alert matches only when route, type, and severity conditions all match.

Subscription behavior

- `subscribe(subscriber, preference)` adds a new subscription; duplicates do not create a second entry.
- `updatePreference(...)` changes an existing subscription; `unsubscribe(...)` stops future delivery.
- New or updated subscribers receive only later matching alerts - stored history is not replayed automatically.
- Publishing the same alert ID twice must not deliver it twice.

Robust notification

- Store alerts in publication history and expose an unmodifiable snapshot.
- Notify from a snapshot of the subscription entries so a callback-time subscription change cannot corrupt the current iteration.
- A commuter maintains an inbox and displays every received alert.

Task

1. Implement the publisher and subscriber abstractions, preference filtering, and duplicate-ID protection.
2. Demonstrate matched and unmatched alerts, a preference update, a duplicate publication, and unsubscription.
3. Keep the transit center dependent on a subscriber abstraction rather than on `Commuter`.

Exam instruction

You may introduce suitable interfaces, classes, enumerations, records, and helper methods. Your solution should be clean, object-oriented, and open to reasonable extension.

SOLUTION | OBSERVER

Design rationale

Why this pattern fits

TransitControl is the subject and maintains observer-preference pairs. It publishes through TransitObserver, not through a concrete commuter type. Preference.matches(...) keeps filtering cohesive, and snapshot iteration makes notification stable.

Pattern-role mapping

Pattern role	Implementation
Subject	TransitControl - subscriptions, history, duplicate IDs, and publication
Observer	TransitObserver - one onAlert(...) callback
Concrete observer	Commuter - displays and stores received messages
Filter value	Immutable Preference - route, type, and minimum-severity predicate

Complete Java solution - TransitAlertDemo.java

```
import java.util.*;

public class TransitAlertDemo {
    enum AlertType { DELAY, CANCELLATION, ROUTE_CHANGE, SAFETY }
    enum Severity { INFO, WARNING, CRITICAL }

    record Alert(
        String id, String route, AlertType type,
        Severity severity, String message) {}

    record Preference(
        Set<String> routes, Set<AlertType> types, Severity minimumSeverity) {
        Preference {
            routes = Set.copyOf(routes);
            types = Set.copyOf(types);
            Objects.requireNonNull(minimumSeverity);
        }

        boolean matches(Alert alert) {
            boolean routeMatch = routes.contains("*") || routes.contains(alert.route());
            boolean typeMatch = types.contains(alert.type());
            boolean severeEnough = alert.severity().ordinal()
                >= minimumSeverity.ordinal();
            return routeMatch && typeMatch && severeEnough;
        }
    }

    interface TransitObserver {
        void onAlert(Alert alert);
    }

    interface TransitSubject {
        void subscribe(TransitObserver observer, Preference preference);
        void updatePreference(TransitObserver observer, Preference preference);
        void unsubscribe(TransitObserver observer);
        void publish(Alert alert);
    }

    static final class TransitControl implements TransitSubject {
        private final Map<TransitObserver, Preference> subscriptions =
            new LinkedHashMap<>();
        private final List<Alert> history = new ArrayList<>();
        private final Set<String> publishedIds = new HashSet<>();

        public void subscribe(TransitObserver observer, Preference preference) {
            Objects.requireNonNull(observer);
            Objects.requireNonNull(preference);
            if (subscriptions.putIfAbsent(observer, preference) != null) {
                System.out.println("Observer is already subscribed; use updatePreference.");
            }
        }

        public void updatePreference(
            TransitObserver observer, Preference preference) {
            Objects.requireNonNull(preference);
            if (!subscriptions.containsKey(observer)) {
                throw new IllegalArgumentException("Observer is not subscribed.");
            }
            subscriptions.put(observer, preference);
        }

        public void unsubscribe(TransitObserver observer) {
            subscriptions.remove(observer);
        }
    }
}
```

```

    public void publish(Alert alert) {
        Objects.requireNonNull(alert);
        if (!publishedIds.add(alert.id())) {
            System.out.println("Duplicate alert " + alert.id() + " ignored.");
            return;
        }
        history.add(alert);
        System.out.println("PUBLISH " + alert.id() + " on " + alert.route());

        // A snapshot prevents callback-time subscription changes from
        // causing ConcurrentModificationException or inconsistent delivery.
        var snapshot = new ArrayList<>(subscriptions.entrySet());
        for (var entry : snapshot) {
            if (entry.getValue().matches(alert)) {
                entry.getKey().onAlert(alert);
            }
        }
    }

    public List<Alert> history() { return List.copyOf(history); }
}

static final class Commuter implements TransitObserver {
    private final String name;
    private final List<String> inbox = new ArrayList<>();

    Commuter(String name) { this.name = name; }

    public void onAlert(Alert alert) {
        String text = alert.severity() + " " + alert.type()
            + " on " + alert.route() + ": " + alert.message();
        inbox.add(text);
        System.out.println(name + " <- " + text);
    }

    public List<String> inbox() { return List.copyOf(inbox); }
}

static Preference preference(
    Set<String> routes, Severity minimum, AlertType... types) {
    return new Preference(routes, EnumSet.copyOf(Arrays.asList(types)), minimum);
}

public static void main(String[] args) {
    TransitControl control = new TransitControl();
    Commuter a = new Commuter("Ayesha");
    Commuter b = new Commuter("Bashir");

    control.subscribe(a, preference(
        Set.of("R1", "R2"), Severity.WARNING,
        AlertType.DELAY, AlertType.CANCELLATION));
    control.subscribe(b, preference(
        Set.of("*"), Severity.CRITICAL,
        AlertType.CANCELLATION, AlertType.SAFETY));

    control.publish(new Alert(
        "A-1", "R1", AlertType.DELAY,
        Severity.INFO, "Five-minute delay"));           // nobody
    control.publish(new Alert(
        "A-2", "R1", AlertType.CANCELLATION,
        Severity.CRITICAL, "Service suspended"));       // both

    control.updatePreference(a, preference(
        Set.of("R3"), Severity.INFO,
        AlertType.DELAY, AlertType.ROUTE_CHANGE));
    control.publish(new Alert(
        "A-3", "R3", AlertType.ROUTE_CHANGE,
        Severity.WARNING, "Use the eastern diversion")); // Ayesha
    control.publish(new Alert(
        "A-3", "R3", AlertType.ROUTE_CHANGE,
        Severity.WARNING, "Duplicate"));                 // ignored

    control.unsubscribe(b);
    System.out.println("Stored alerts: " + control.history().size());
}
}

```

Implementation check

This listing is self-contained and uses one public demo class. Save it under the filename `TransitAlertDemo.java`, then compile and run it with Java 17 or later.

Quick Pattern Selection Guide

A useful exam habit

Do not choose a pattern from nouns in the story. First identify the design pressure: changing condition, interchangeable algorithm, tangled communication, invariant workflow, or one-to-many notification.

When the main difficulty is...	Prefer	Key ownership
An object's valid behavior changes with its condition	State	Each state owns condition-specific behavior and transitions
One algorithm must be replaceable at runtime	Strategy	Each strategy owns one algorithm
Many participants need centralized interaction rules	Mediator	Mediator owns coordination; colleagues stay decoupled
The workflow order is fixed but some steps vary	Template Method	Base class owns the skeleton; subclasses fill steps
Many dependents react to publisher events	Observer	Subject stores observers; observers own their reactions

End of practice set. Reattempt the five problems without looking at the pattern badges to practice pattern identification under exam conditions.